



Manual de práctica

Gestión de una base de datos

M3. Bases de datos



Introducción

El uso de Docker, PostgreSQL y Python en el desarrollo de software, es clave para crear aplicaciones escalables, eficientes y portables. En esta práctica aprenderás a crear una base de datos de forma automática dentro de un contenedor, utilizando docker-compose. Con el empleo del archivo dockerfile, se definirá la estructura inicial de tablas y relaciones. Además, agregarás los registros iniciales de tu base de datos. Finalmente, realizarás las primeras consultas a tu base de datos utilizando Python.

Recursos necesarios



- Windows 10 u 11, x86_64, 22H2 o superior
- 4GB de RAM
- WSL 2
- Virtualización habilitada en BIOS



- Docker Desktop
- Sublime Text
- Python

Etapas

Antes de iniciar, es necesario que identifiques las etapas que integran esta práctica.



Desarrollo de la práctica



Notas

Esta práctica está planeada únicamente para llevarse a cabo en el **sistema operativo Windows**.

Es posible que los comandos y la instalación de las herramientas tengan diferencias con Linux y MacOS, por tanto, es recomendable que te prepares con antelación, para evitar contratiempos.

Etapa 1. Creación de la base de datos

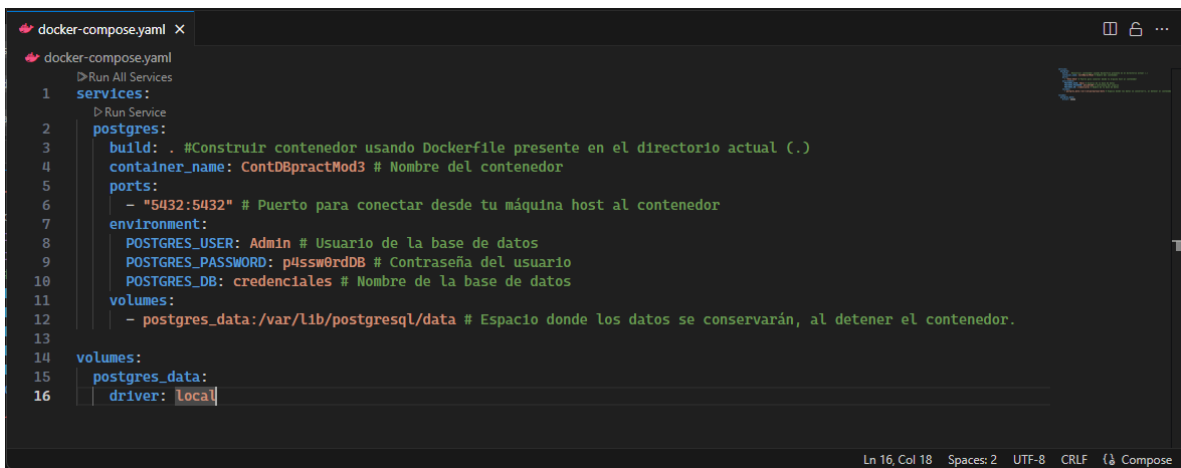
1. En el escritorio de tu computadora, **crea** una nueva carpeta con el nombre **practicaMod3**.
2. Dentro de la carpeta **practicaMod3**, **crea** otra carpeta con el nombre **DBconfiguration**.
3. **Inicia** el editor *Sublime Text*.
4. Una vez iniciado el programa, **crea** un archivo nombrado **docker-compose** que contenga la siguiente información:

docker-compose.yml

```
services:
  postgres:
    build: . #Construir contenedor usando Dockerfile presente en el
    directorio actual (.)
    container_name: ContDBpractMod3 # Nombre del contenedor
    ports:
      - "5432:5432" # Puerto para conectar desde tu máquina host al
    contenedor
    environment:
      POSTGRES_USER: Admin # Usuario de la base de datos
      POSTGRES_PASSWORD: p4ssw0rdDB # Contraseña del usuario
      POSTGRES_DB: credenciales # Nombre de la base de datos
    volumes:
      - postgres_data:/var/lib/postgresql/data # Espacio donde los datos
    se conservarán, al detener el contenedor.

volumes:
  postgres_data:
    driver: local
```

A continuación, te dejamos una imagen para que puedas revisar con detalle cómo debe quedar la información.

A screenshot of a code editor window titled 'docker-compose.yml'. The editor shows the same YAML configuration as the previous block, with line numbers 1 through 16 on the left margin. The code is syntax-highlighted, with keywords in blue and comments in green. The editor interface includes a top bar with icons for file operations and a bottom status bar showing 'Ln 16, Col 18', 'Spaces: 2', 'UTF-8', 'CRLF', and a 'Compose' button.



Notas

En archivos de configuración YAML y en scripts de Python, las **sangrías** son fundamentales. Estos lenguajes utilizan la indentación para definir jerarquías y bloques de código, por lo que un espacio extra o faltante puede causar errores difíciles de detectar.

¡Asegúrate siempre de revisar bien las sangrías al copiar y pegar código!

5. **Guarda** el archivo con extensión **.YAML** en la carpeta **DBconfiguration**.



Notas

No agregues extensiones al nombre del archivo. Al guardarlo, selecciona **“YAML”** en el campo de tipo de archivo.

Nombre:

Tipo:

6. Con ayuda del editor *Sublime Text* **crea** un archivo nombrado **Dockerfile** que contenga la siguiente información:

Dockerfile.All Files (*.*)

```
# Usa la imagen oficial de PostgreSQL
FROM postgres:latest

# Copia el script SQL al directorio de inicialización de PostgreSQL
COPY init.sql /docker-entrypoint-initdb.d/
COPY save_data.sql /docker-entrypoint-initdb.d/

# PostgreSQL ejecutará automáticamente los scripts en /docker-entrypoint-initdb.d/
```

7. **Guarda** el archivo con extensión **.All files(*.*)** en la carpeta **DBconfiguration**.



Notas

No agregues extensiones al nombre del archivo. Al guardarlo, selecciona "All Files (*.*)" en el campo de tipo de archivo.

Nombre: Dockerfile

Tipo: All Files (*.*)

8. Con ayuda del editor *Sublime Text* **crea** un archivo nombrado **save_data** que contenga la siguiente información:

save_data.sql

```
-- Insertar datos en la tabla usuarios
INSERT INTO usuarios (nombre, correo, telefono, fecha_nacimiento)
VALUES
('Juan Pérez', 'juan.perez1@example.com', '1234567890', '1985-01-15'),
('Ana Gómez', 'ana.gomez2@example.com', '1234567891', '1990-03-22'),
('Luis Martínez', 'luis.martinez3@example.com', '1234567892', '1988-07-10'),
('María López', 'maria.lopez4@example.com', '1234567893', '1992-11-05'),
('Carlos Ruiz', 'carlos.ruiz5@example.com', '1234567894', '1980-06-25'),
('Sofía Castro', 'sofia.castro6@example.com', '1234567895', '1995-02-18'),
('David Ramírez', 'david.ramirez7@example.com', '1234567896', '1983-09-09'),
('Elena Ortega', 'elena.ortega8@example.com', '1234567897', '1991-05-03'),
('Miguel Torres', 'miguel.torres9@example.com', '1234567898', '1993-12-14'),
('Laura Sánchez', 'laura.sanchez10@example.com', '1234567899', '1987-08-01'),
('Pedro Morales', 'pedro.morales11@example.com', '1234567800', '1986-04-17'),
('Clara Hernández', 'clara.hernandez12@example.com', '1234567801', '1994-06-30'),
('Jorge Rojas', 'jorge.rojas13@example.com', '1234567802', '1981-03-25'),
('Valeria Peña', 'valeria.pena14@example.com', '1234567803', '1992-01-09'),
('Andrés Romero', 'andres.romero15@example.com', '1234567804', '1989-10-12'),
('Camila Paredes', 'camila.paredes16@example.com', '1234567805', '1997-11-19'),
('Ricardo Vargas', 'ricardo.vargas17@example.com', '1234567806', '1984-07-23'),
('Daniela Flores', 'daniela.flores18@example.com', '1234567807', '1996-03-01'),
('Héctor Serrano', 'hector.serrano19@example.com', '1234567808', '1982-02-11'),
('Patricia Vega', 'patricia.vega20@example.com', '1234567809', '1990-09-05');
```

```
-- Insertar datos en la tabla credenciales
INSERT INTO credenciales (id_usuario, username, password_hash)
VALUES
(1, 'juan.perez1', 'hash_juan_perez'),
(2, 'ana.gomez2', 'hash_ana_gomez'),
(3, 'luis.martinez3', 'hash_luis_martinez'),
(4, 'maria.lopez4', 'hash_maria_lopez'),
(5, 'carlos.ruiz5', 'hash_carlos_ruiz'),
(6, 'sofia.castro6', 'hash_sofia_castro'),
(7, 'david.ramirez7', 'hash_david_ramirez'),
(8, 'elena.ortega8', 'hash_elena_ortega'),
(9, 'miguel.torres9', 'hash_miguel_torres'),
(10, 'laura.sanchez10', 'hash_laura_sanchez'),
(11, 'pedro.morales11', 'hash_pedro_morales'),
(12, 'clara.hernandez12', 'hash_clara_hernandez'),
(13, 'jorge.rojas13', 'hash_jorge_rojas'),
(14, 'valeria.pena14', 'hash_valeria_pena'),
(15, 'andres.romero15', 'hash_andres_romero'),
(16, 'camila.paredes16', 'hash_camila_paredes'),
(17, 'ricardo.vargas17', 'hash_ricardo_vargas'),
(18, 'daniela.flores18', 'hash_daniela_flores'),
(19, 'hector.serrano19', 'hash_hector_serrano'),
(20, 'patricia.vega20', 'hash_patricia_vega');
```

9. **Guarda** el archivo con extensión **.SQL** en la carpeta **DBconfiguration**.



Notas

No agregues extensiones al nombre del archivo. Al guardarlo, selecciona **"SQL"** en el campo de tipo de archivo.

Nombre: save_data.sql

Tipo: SQL (*.sql;*.ddl;*.dml)

10. Con ayuda del editor *Sublime Text* **crea** un archivo nombrado **init** que contenga la siguiente información:

init.sql

```
-- Crear la tabla para almacenar datos de usuarios
CREATE TABLE usuarios (
  id_usuario SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  correo VARCHAR(255) UNIQUE,
  telefono VARCHAR(15),
  fecha_nacimiento DATE
);

-- Crear la tabla para almacenar usuarios y contraseñas
CREATE TABLE credenciales (
  id_credencial SERIAL PRIMARY KEY,
  id_usuario INT NOT NULL,
  username VARCHAR(50) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  FOREIGN KEY (id_usuario) REFERENCES usuarios (id_usuario)
);
```

11. **Guarda** el archivo con extensión **.SQL** en la carpeta **DBconfiguration**.



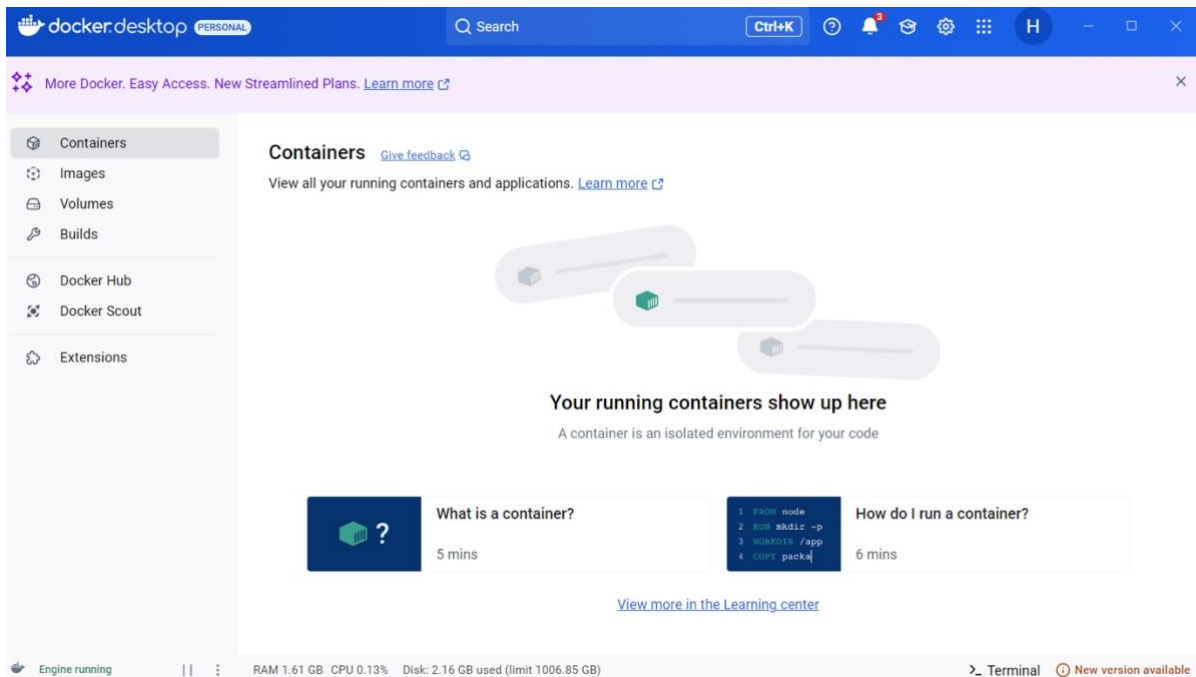
Notas

No agregues extensiones al nombre del archivo. Al guardarlo, selecciona **"SQL"** en el campo de tipo de archivo.

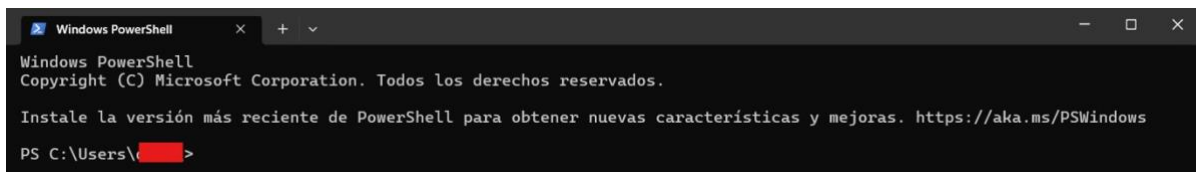
Nombre:

Tipo:

12. **Inicia** el programa *Docker Desktop*.



13. **Inicia** una terminal de *Windows PowerShell*.



14. **Cambia** la ubicación de la terminal a la carpeta **DBconfiguration**.

Comando de ejemplo:

```
cd .\Desktop\practicaMod3\DBconfiguration\
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\[redacted]> cd .\Desktop\practicaMod3\DBconfiguration\
PS C:\Users\[redacted]\Desktop\practicaMod3\DBconfiguration>
```

En tu computadora, la carpeta **Desktop** puede aparecer como **Escritorio**. Para revisar el contenido de la ruta en la que te encuentras, puedes usar el comando **ls**. Si deseas ingresar a una carpeta, utiliza el comando **cd**, y para volver a la carpeta anterior, usa la instrucción **cd ..**



Notas

```
Windows PowerShell
PS C:\Users\[redacted]\Desktop\practicaMod3\DBconfiguration> cd ..
PS C:\Users\[redacted]\Desktop\practicaMod3> cd ..
PS C:\Users\[redacted]\Desktop> cd ..
PS C:\Users\[redacted]> ls

Directorio: C:\Users\[redacted]
```

Mode	LastWriteTime	Length	Name
d-----	27/01/2025 10:53 p. m.		.aws
d-----	27/01/2025 10:53 p. m.		.azure
d-----	12/01/2022 04:50 p. m.		.cache
d-----	29/11/2024 12:04 p. m.		.config
d-----	28/01/2025 05:58 a. m.		.docker
d-----	26/11/2024 03:25 p. m.		.eclipse
d-----	13/04/2022 12:39 a. m.		.gradle
d-----	20/03/2022 07:20 p. m.		.m2
d-----	25/06/2022 10:18 p. m.		.ms-ad
d-----	20/03/2022 07:16 p. m.		.nbi
d-----	29/11/2024 12:54 p. m.		.p2
d-----	26/11/2024 03:43 p. m.		.redhat
d-----	23/06/2023 01:17 p. m.		.ssh
d-----	03/11/2023 01:28 a. m.		.swt
d-----	21/06/2023 02:42 p. m.		.vscode
d-r----	28/12/2020 01:46 p. m.		3D Objects
d-----	13/04/2022 02:53 a. m.		AndroidStudioProjects
d-----	29/12/2024 02:37 p. m.		ansel
d-r----	25/01/2025 05:41 p. m.		Contacts
d-r----	28/01/2025 05:20 a. m.		Desktop
d-r----	25/01/2025 05:41 p. m.		Documents
d-r----	30/01/2025 12:26 a. m.		Downloads
d-----	20/03/2022 07:03 p. m.		eclipse
d-----	29/11/2024 12:54 p. m.		eclipse-workspace

15. Inmediatamente, **ejecuta** el comando `docker-compose up -d` para crear el contenedor y la base de datos de forma automática.

Comando:

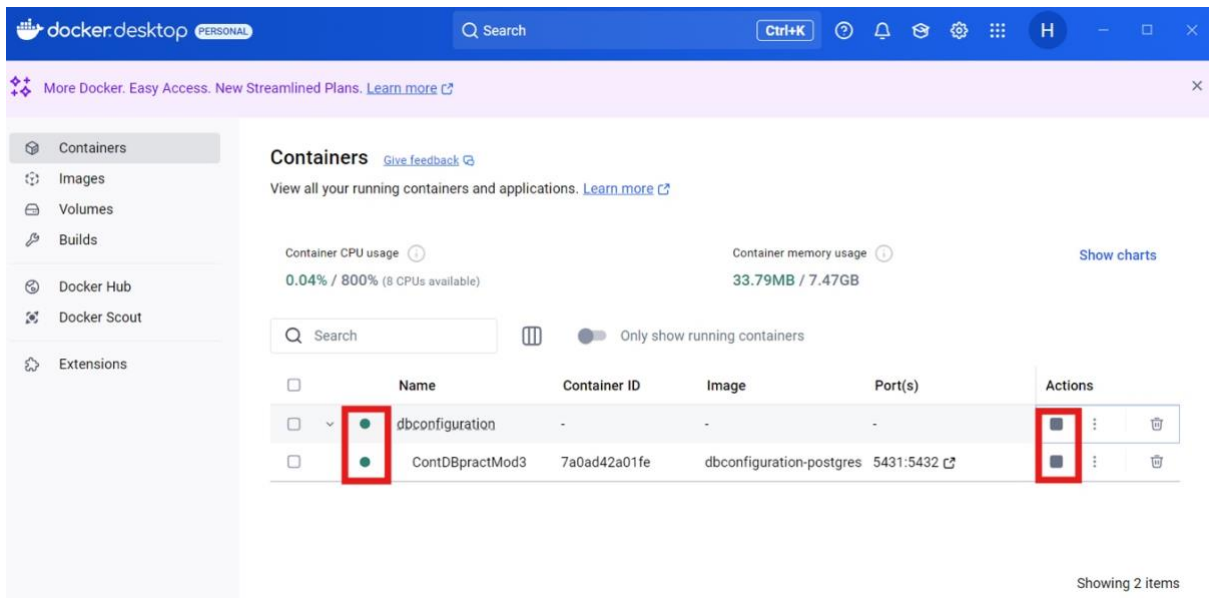
```
docker-compose up -d
```

```
PS C:\Users\█████\Desktop\practicaMod3\DBconfiguration> docker-compose up -d
```

La terminal aparecerá de la siguiente manera:

```
PS C:\Users\█████\Desktop\practicaMod3\DBconfiguration> docker-compose up -d
[+] Running 0/0
[+] Running 0/1gres Building
[+] Building 3.6s (10/10) FINISHED
=> [postgres internal] load build definition from Dockerfile
=> => transferring dockerfile: 351B
=> [postgres internal] load metadata for docker.io/library/postgres:latest
=> [postgres auth] library/postgres:pull token for registry-1.docker.io
=> [postgres internal] load .dockerignore
=> => transferring context: 2B
=> [postgres 1/3] FROM docker.io/library/postgres:latest@sha256:87ec5e0a167dc7d4831729f9e1d2ee7b8597dcc49ccd9e43
=> => resolve docker.io/library/postgres:latest@sha256:87ec5e0a167dc7d4831729f9e1d2ee7b8597dcc49ccd9e43cc5f89e80
=> [postgres internal] load build context
=> => transferring context: 63B
=> CACHED [postgres 2/3] COPY init.sql /docker-entrypoint-initdb.d/
=> CACHED [postgres 3/3] COPY save_data.sql /docker-entrypoint-initdb.d/
=> [postgres] exporting to image
=> => exporting layers
=> => exporting manifest sha256:66d1d630a7a11f092ff0b737dc5deefb50a0e09f17be2970b4bc44300d9a154f
=> => exporting config sha256:c661a0b847e1f18c67f88850341e7e098471e0da6a434283f22262e883fe7c5c
=> => exporting attestation manifest sha256:13d545e1b5d072f8dae281aaaf1a8eedbdf54762032a65a3e08e730f59ebac5
=> => exporting manifest list sha256:41c08f4a58b20603f4687ed99f94f651ad88ac13a253b6b14e689a1ba158c509
=> => naming to docker.io/library/dbconfiguration-postgres:latest
[+] Running 4/4g to docker.io/library/dbconfiguration-postgres:latest
✔Service postgres Built
✔Network dbconfiguration_default Created
✔Volume "dbconfiguration-postgres_data" Created
✔Container ContDBpractMod3 Started
PS C:\Users\█████\Desktop\practicaMod3\DBconfiguration>
```

16. **Verifica** que el contenedor esté activo en *Docker Desktop*. Si el círculo al lado del nombre del contenedor es de color **verde**, significa que el contenedor está **activo**.



Notas

Si se ejecutó el comando del paso 15 y el contenedor no aparece o aparece inactivo, te recomendamos revisar nuevamente el proceso de instalación.

Puedes consultar los apartados de la sección *para saber más*, para mayor información sobre activar o desactivar los contenedores, o bien, acércate a tu facilitador, en caso de tener más dudas.

17. Valida que tu base de datos se haya creado correctamente, ejecutando los siguientes comandos en Windows PowerShell:

- a. `docker exec -it ContDBpractMod3 bash`
- b. `psql -U Admin -d credenciales`
- c. `SELECT * FROM usuarios;`

Resultado esperado:

```
PS C:\Users\██████\Desktop\practicaMod3\DBconfiguration> docker exec -it ContDBpractMod3 bash
root@8a72dae463c3:/# psql -U Admin -d credenciales
psql (17.2 (Debian 17.2-1.pgdg120+1))
Type "help" for help.

credenciales=# SELECT * FROM usuarios;
 id_usuario | nombre          | correo                      | telefono | fecha_nacimiento
-----+-----+-----+-----+-----
          1 | Juan Pérez      | juan.perez1@example.com    | 1234567890 | 1985-01-15
          2 | Ana Gómez       | ana.gomez2@example.com     | 1234567891 | 1990-03-22
          3 | Luis Martínez   | luis.martinez3@example.com  | 1234567892 | 1988-07-10
          4 | María López     | maria.lopez4@example.com    | 1234567893 | 1992-11-05
          5 | Carlos Ruiz     | carlos.ruiz5@example.com    | 1234567894 | 1980-06-25
          6 | Sofía Castro    | sofia.castro6@example.com   | 1234567895 | 1995-02-18
          7 | David Ramírez   | david.ramirez7@example.com  | 1234567896 | 1983-09-09
          8 | Elena Ortega    | elena.ortega8@example.com   | 1234567897 | 1991-05-03
          9 | Miguel Torres   | miguel.torres9@example.com  | 1234567898 | 1993-12-14
         10 | Laura Sánchez   | laura.sanchez10@example.com | 1234567899 | 1987-08-01
         11 | Pedro Morales   | pedro.morales11@example.com | 1234567800 | 1986-04-17
         12 | Clara Hernández | clara.hernandez12@example.com | 1234567801 | 1994-06-30
         13 | Jorge Rojas     | jorge.rojas13@example.com   | 1234567802 | 1981-03-25
         14 | Valeria Peña    | valeria.pena14@example.com  | 1234567803 | 1992-01-09
         15 | Andrés Romero   | andres.romero15@example.com  | 1234567804 | 1989-10-12
         16 | Camila Paredes  | camila.paredes16@example.com | 1234567805 | 1997-11-19
         17 | Ricardo Vargas  | ricardo.vargas17@example.com | 1234567806 | 1984-07-23
         18 | Daniela Flores  | daniela.flores18@example.com | 1234567807 | 1996-03-01
         19 | Héctor Serrano  | hector.serrano19@example.com | 1234567808 | 1982-02-11
         20 | Patricia Vega   | patricia.vega20@example.com  | 1234567809 | 1990-09-05
(20 rows)

credenciales=#
```

Si al ejecutar los comandos obtienes mensajes de **error**, se recomienda revisar el proceso de instalación.

Para salir de PostgreSQL y regresar al modo normal de tu terminal de *PowerShell*, puedes ejecutar el comando **exit dos veces**, como se muestra en la siguiente imagen:


```

root@7a0ad42a01fe:/# psql -U Admin -d credenciales
psql (17.2 (Debian 17.2-1.pgdg120+1))
Type "help" for help.

credenciales=# SELECT * FROM usuarios;
 id_usuario | nombre      | correo                      | telefono | fecha_nacimiento
-----+-----+-----+-----+-----
      1 | Juan Pérez | juan.perez1@example.com    | 1234567890 | 1985-01-15
      2 | Ana Gómez  | ana.gomez2@example.com    | 1234567891 | 1990-03-22
      3 | Luis Martínez | luis.martinez3@example.com | 1234567892 | 1988-07-10
      4 | María López | maria.lopez4@example.com   | 1234567893 | 1992-11-05
      5 | Carlos Ruiz | carlos.ruiz5@example.com   | 1234567894 | 1980-06-25
      6 | Sofía Castro | sofia.castro6@example.com  | 1234567895 | 1995-02-18
      7 | David Ramírez | david.ramirez7@example.com | 1234567896 | 1983-09-09
      8 | Elena Ortega | elena.ortega8@example.com  | 1234567897 | 1991-05-03
      9 | Miguel Torres | miguel.torres9@example.com | 1234567898 | 1993-12-14
     10 | Laura Sánchez | laura.sanchez10@example.com | 1234567899 | 1987-08-01
     11 | Pedro Morales | pedro.morales11@example.com | 1234567800 | 1986-04-17
     12 | Clara Hernández | clara.hernandez12@example.com | 1234567801 | 1994-06-30
     13 | Jorge Rojas | jorge.rojas13@example.com  | 1234567802 | 1981-03-25
     14 | Valeria Peña | valeria.pena14@example.com | 1234567803 | 1992-01-09
     15 | Andrés Romero | andres.romero15@example.com | 1234567804 | 1989-10-12
     16 | Camila Paredes | camila.paredes16@example.com | 1234567805 | 1997-11-19
     17 | Ricardo Vargas | ricardo.vargas17@example.com | 1234567806 | 1984-07-23
     18 | Daniela Flores | daniela.flores18@example.com | 1234567807 | 1996-03-01
     19 | Héctor Serrano | hector.serrano19@example.com | 1234567808 | 1982-02-11
     20 | Patricia Vega | patricia.vega20@example.com | 1234567809 | 1990-09-05

(20 rows)

credenciales=# exit
root@7a0ad42a01fe:/# exit
exit
PS C:\Users\... \Desktop\practicaMod3\DBconfiguration>

```

¿Cómo puedo **activar/desactivar** mi contenedor?

Tienes dos opciones:

1. Presionando el símbolo **Stop/Start** desde la interfaz de docker desktop en la columna **Actions**.



☐	Name	Container ID	Image	Port(s)	Actions
☐	dbconfiguration	-	-	-	  
☐	ContDBpractMod3	7a0ad42a01fe	dbconfiguration-postgres	5431:5432	  

2. Desde una terminal, ingresando los siguientes comandos:

- docker-compose start
- docker-compose stop



Notas

- Recuerda que siempre que quieras acceder a la base de datos, el contenedor debe estar activo.
- Puedes detener y eliminar el contenedor desde *Docker Desktop* o desde la terminal de *PowerShell*.



Para saber más

En el archivo `docker-compose.yaml` se encuentran las instrucciones de configuración del contenedor.

La instrucción `build` indica que se utilizarán instrucciones adicionales, que se encuentran en el archivo `Dockerfile`; en este caso, se copiarán los archivos `init.sql` y `save_data.sql` desde nuestra máquina local al contenedor.

Cuando el contenedor se inicializa, se ejecutan los archivos `init.sql` y `save_data.sql`. El primero generará las tablas de nuestra base de datos, y el segundo insertará los registros en las tablas.

Etapa 2. Transacciones a la Base de Datos

1. Con ayuda del editor *Sublime Text* **crea** un archivo nombrado **acceso** que contenga la siguiente información:

acceso.py

```
import psycopg2
import getpass

# Configuración de conexión a la base de datos en Docker
DB_HOST = "localhost"
DB_PORT = "5432"
DB_NAME = "credenciales"
DB_USER = 'Admin'
DB_PASSWORD = "p4ssw0rdDB"

def conectar_db():
    """Conecta a la base de datos PostgreSQL y retorna la conexión."""
    try:
        conn = psycopg2.connect(
            host=DB_HOST,
            port=DB_PORT,
            database=DB_NAME,
            user=DB_USER,
            password=DB_PASSWORD
        )
        return conn
    except Exception as e:
        print(e)
        #print("Error de conexión a la base de datos:", e)
        return None

def obtener_datos_usuario(username, password):
    """Consulta la base de datos para obtener los datos de un usuario a partir de sus credenciales."""
    conn = conectar_db()
    if not conn:
```



```
    return

    try:
        cursor = conn.cursor()

        # Verificar si el usuario y contraseña existen en la tabla
        credenciales
        query = """
        SELECT u.id_usuario, u.nombre, u.correo, u.telefono,
        u.fecha_nacimiento
        FROM credenciales c
        JOIN usuarios u ON c.id_usuario = u.id_usuario
        WHERE c.username = %s AND c.password_hash = %s;
        """

        cursor.execute(query, (username, password))
        usuario = cursor.fetchone()

        if usuario:
            print("\nDatos del usuario encontrado:")
            print(f"ID: {usuario[0]}")
            print(f"Nombre: {usuario[1]}")
            print(f"Correo: {usuario[2]}")
            print(f"Teléfono: {usuario[3]}")
            print(f"Fecha de Nacimiento: {usuario[4]}")
        else:
            print("\nUsuario o contraseña incorrectos.")

        cursor.close()
        conn.close()

    except Exception as e:
        print("Error al consultar la base de datos:", e)

if __name__ == "__main__":
    print("Inicio de sesión en la base de datos")

    # Solicitar credenciales al usuario
    username = input("Ingrese su usuario: ")
    password = getpass.getpass("Ingrese su contraseña: ") # No muestra
    la contraseña al escribir
```

```
# Consultar la base de datos
obtener_datos_usuario(username, password)
```

A continuación, te dejamos una imagen para que puedas revisar con detalle cómo debe quedar la información.

```

1 import pygame
2 import pygame
3
4 # Configuración de conexión a la base de datos en Docker
5 DB_HOST = "localhost"
6 DB_PORT = "5432"
7 DB_NAME = "credenciales"
8 DB_USER = "admin"
9 DB_PASSWORD = "password"
10
11 def conectar_db():
12     """Conecta a la base de datos PostgreSQL y retorna la conexión."""
13     try:
14         conn = psycopg2.connect(
15             host=DB_HOST,
16             port=DB_PORT,
17             database=DB_NAME,
18             user=DB_USER,
19             password=DB_PASSWORD
20         )
21         return conn
22     except Exception as e:
23         print(e)
24         print("Error de conexión a la base de datos:", e)
25         return None
26
27 def obtener_datos_usuario(username, password):
28     """Consulta la base de datos para obtener los datos de un usuario a partir de sus credenciales."""
29     conn = conectar_db()
30     if not conn:
31         return
32
33     try:
34         cursor = conn.cursor()
35
36         # Verificar si el usuario y contraseña existen en la tabla credenciales
37         query = """
38             SELECT u.id_usuario, u.nombre, u.correo, u.telefono, u.fecha_nacimiento
39             FROM credenciales c
40             JOIN usuarios u ON c.id_usuario = u.id_usuario
41             WHERE c.username = %s AND c.password_hash = %s;
42         """
43         cursor.execute(query, (username, password))
44         usuario = cursor.fetchone()
45
46         if usuario:
47             print("\nDatos del usuario encontrados:")
48             print(f"ID: {usuario[0]}")
49             print(f"Nombre: {usuario[1]}")
50             print(f"Correo: {usuario[2]}")
51             print(f"Teléfono: {usuario[3]}")
52             print(f"Fecha de nacimiento: {usuario[4]}")
53         else:
54             print("\nUsuario o contraseña incorrectos.")
55
56         cursor.close()
57         conn.close()
58
59     except Exception as e:
60         print("Error al consultar la base de datos:", e)
61
62 if __name__ == "__main__":
63     print("Inicio de sesión en la base de datos")
64
65     # Solicitar credenciales al usuario
66     username = input("Ingrese su usuario: ")
67     password = getpass.getpass("Ingrese su contraseña: ") # Se muestra la contraseña al escribir
68
69     # Consultar la base de datos
70     obtener_datos_usuario(username, password)
71

```



Notas

En archivos de configuración YAML y en scripts de Python, las **sangrías** son fundamentales. Estos lenguajes utilizan la indentación para definir jerarquías y bloques de código, por lo que un espacio extra o faltante puede causar errores difíciles de detectar.

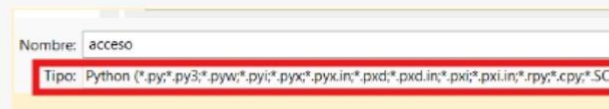
¡Asegúrate siempre de revisar bien las sangrías al copiar y pegar código!

2. **Guarda** el archivo con extensión **.py** en la carpeta **DBconfiguration**.



Notas

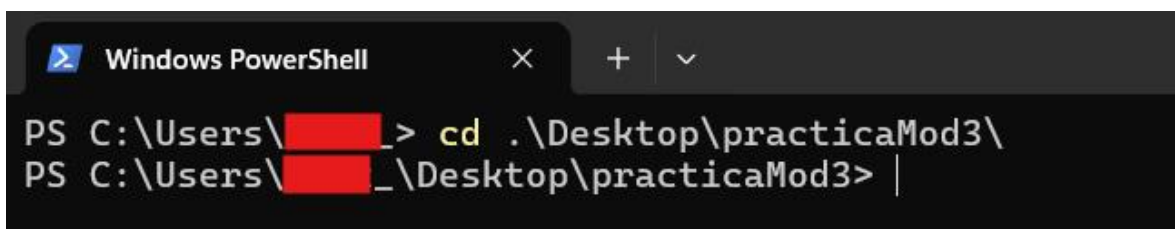
No agregues extensiones al nombre del archivo. Al guardarlo, selecciona "Python" en el campo de tipo de archivo.



3. **Inicia** una terminal de *Windows PowerShell*.
4. **Cambia** la ubicación de la terminal a la carpeta **DBconfiguration**.

Comando de ejemplo:

```
cd .\Desktop\practicaMod3\
```

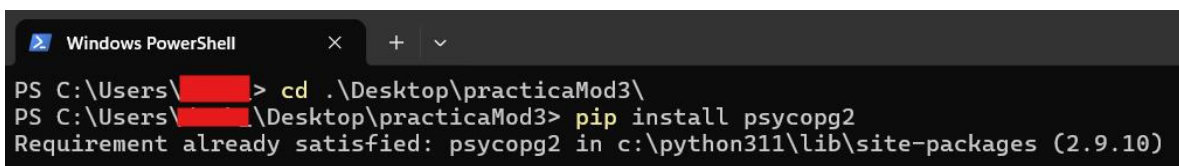


```
Windows PowerShell
PS C:\Users\[redacted]> cd .\Desktop\practicaMod3\
PS C:\Users\[redacted]\Desktop\practicaMod3> |
```

5. **Instala** la librería *psycpg2* ejecutando el comando *pip* en la terminal de *Windows PowerShell*.

Comando de instalación:

```
pip install psycpg2
```



```
Windows PowerShell
PS C:\Users\[redacted]> cd .\Desktop\practicaMod3\
PS C:\Users\[redacted]\Desktop\practicaMod3> pip install psycpg2
Requirement already satisfied: psycpg2 in c:\python311\lib\site-packages (2.9.10)
```

6. Ejecuta el archivo *acceso.py* utilizando *Python 3* ingresando el siguiente comando:

Comando de ejecución en *Windows PowerShell*:

```
python .\acceso.py
```



Notas

Este programa establece una conexión con la base de datos que se encuentra en el contenedor, y solicita un nombre de usuario y contraseña que ya existen en la base de datos. Si ingresamos el nombre de usuario *juan.perez1* y la contraseña *hash_juan_perez*, el programa debería devolver una respuesta como la siguiente:

```
Datos del usuario encontrado:
ID: 1
Nombre: Juan Pérez
Correo: juan.perez1@example.com
Teléfono: 1234567890
Fecha de Nacimiento: 1985-01-15
```



Ejercicio de reforzamiento

Prueba acceder utilizando más nombres de usuario y contraseñas. Puedes consultar todos los usuarios y contraseñas válidos en el archivo `save_data.sql`.

Intenta agregar una **función** al programa que te permita **registrar nuevos usuarios y contraseñas** en la base de datos.



Ejercicio de reforzamiento

Si quieres seguir practicando, muestra en la terminal todos los elementos de la tabla *credenciales*:

1. Notarás que la base de datos tiene el mismo nombre que una de sus tablas, 'credenciales'. Este problema puede afectar la consistencia y la navegabilidad. **¿Cómo puedes solucionar este problema de diseño?**
2. Imagina que esta base de datos se utilizará para el control de trabajadores de una institución. **Agrega una tabla con la información relacionada a sus puestos de trabajo y asóciala al registro del personal.** Si es necesario, realizar ajustes en el diseño de la base de datos.

¡Listo!

Has concluido exitosamente la práctica sobre Gestión de bases de datos.